

반려인-시터 매칭 서비스 FOR-PETS

집사조일체

최길중 | 권지원 | 박영수 | 선경안 | 이지민

배포

<https://forpetscare.uk>



스파르타 내일배움캠프
최종 프로젝트

GitHub

<https://github.com/Housekeeper-for-pets>

✓ 목차

프로젝트 소개

주제 선정 배경
서비스 차별점
팀 소개
개발 일정

01

핵심 기능 구현

주요 기능
동시성 제어
성능 개선
AI 기능

03

설계 및 인프라

ERD
API 설계
AWS 아키텍처
개발 환경

02

검증 및 회고

테스트 결과
트러블슈팅
회고
Q&A

04

프로젝트 소개

주제 선정 배경

서비스 차별점

팀 소개

개발 일정

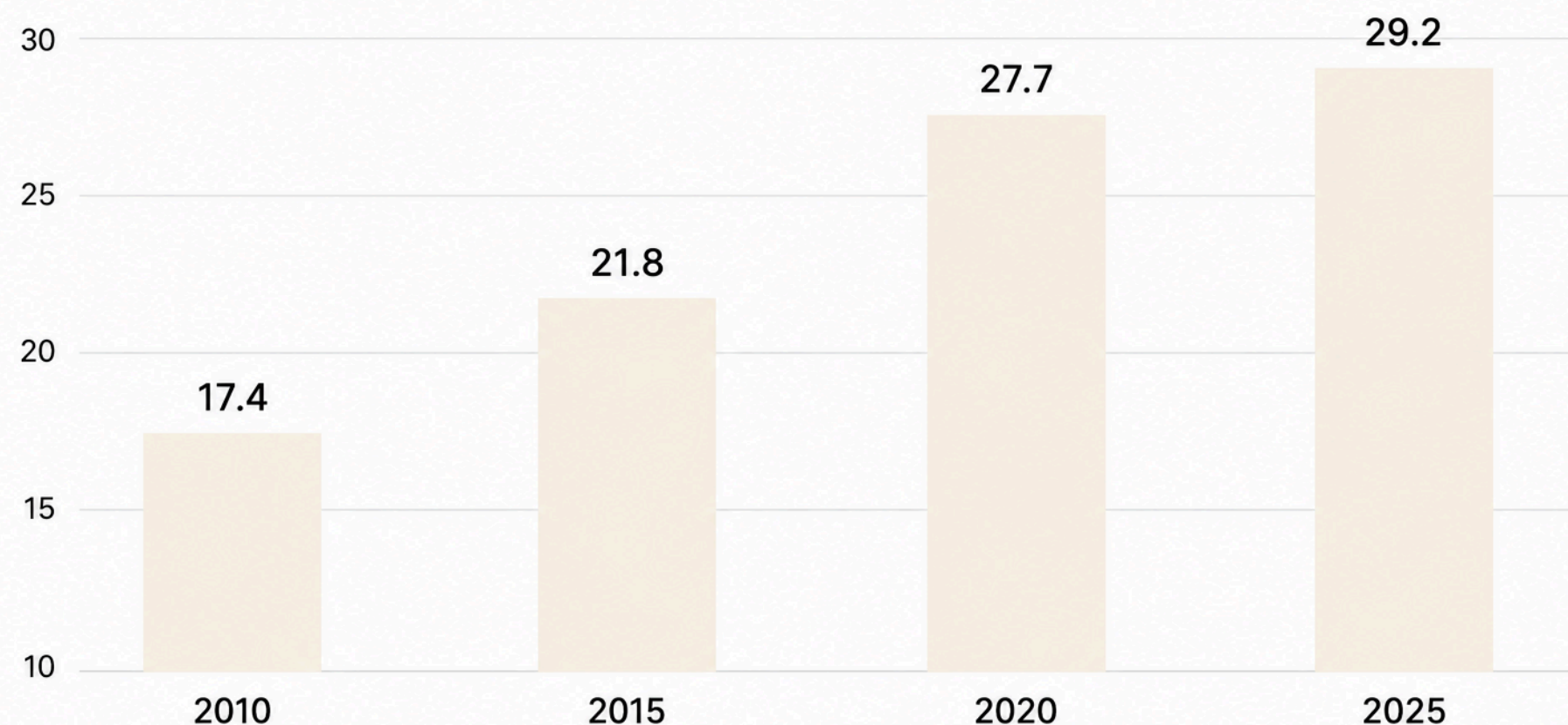


주제 선정 배경

2010 ~ 2025 국내 반려동물 양육비율

자료 : 농림축산식품부

(단위 : %)



2010년: 전화조사(2천명)

2015년: 전화조사/온라인 조사 병행 (3천명)

2020년: 온라인조사 (5천명)

2025년: 면접 조사 (3천가구) - 반려동물 양육 현황 조사 (국가 승인 통계)

국내 반려동물 양육 가구 비율 역대 최대 29.2%

- 1~2인 가구 및 맞벌이 가구 증가에 따른 반려동물 돌봄이 필요한 가구 확대
- 전문 펫시터의 돌봄 서비스 수요 증가
- 반려동물 시장 규모 증가
(2025년 3조 원 → 2027년 6조 원 전망)

출처: 한국 농촌 경제 연구원(KREI)

✓ 기존 서비스 분석 및 차별점

✓ 기존 서비스 분석

국내 펫시터 매칭 서비스 : **순방향 구조** 중심

- 보호자가 직접 돌봄 요청 → 후기 · 프로필 · 비용 비교 부담
- 예약 이후 소통 · 결제 · 정산 분절

✓ ForPets 차별점

역방향 매칭

시터 → 보호자
공고에 직접 제안
보호자의 검색 부담 해소

AI 기반 선택 보조

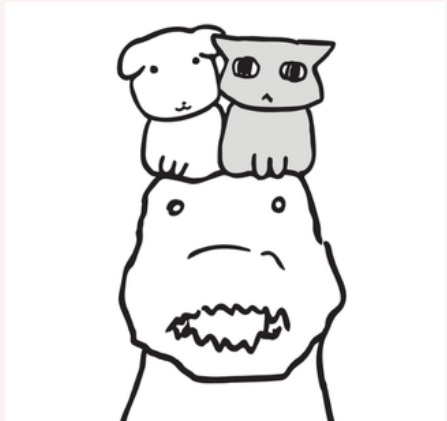
리뷰 요약 AI
시터 추천 AI
→ 비교 비용 절감

통합 예약 관리

예약 - 쿠폰 - 결제
정산 - 채팅 - 케어일지
단일 흐름



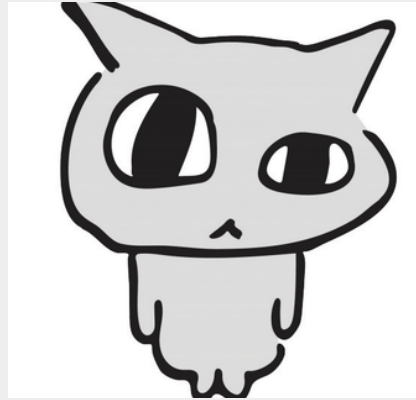
팀원 소개



최길중

리더
인증/인가, 채팅, 쿠폰 동시성

충땅



권지원

메인 비즈니스 로직
스케줄링, 동시성, 모니터링

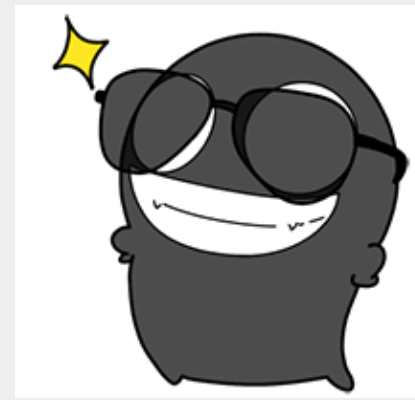
4랑하는
4조



박영수

AWS 인프라,
CI/CD, 실시간 알림

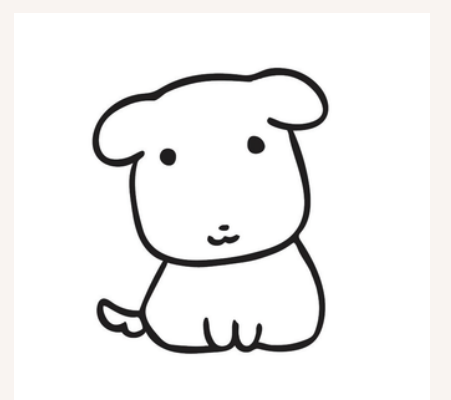
불태웠다..
오얼



선경안

조회 성능개선(캐싱/인덱싱)
리뷰, 모니터링

함께라면



이지민

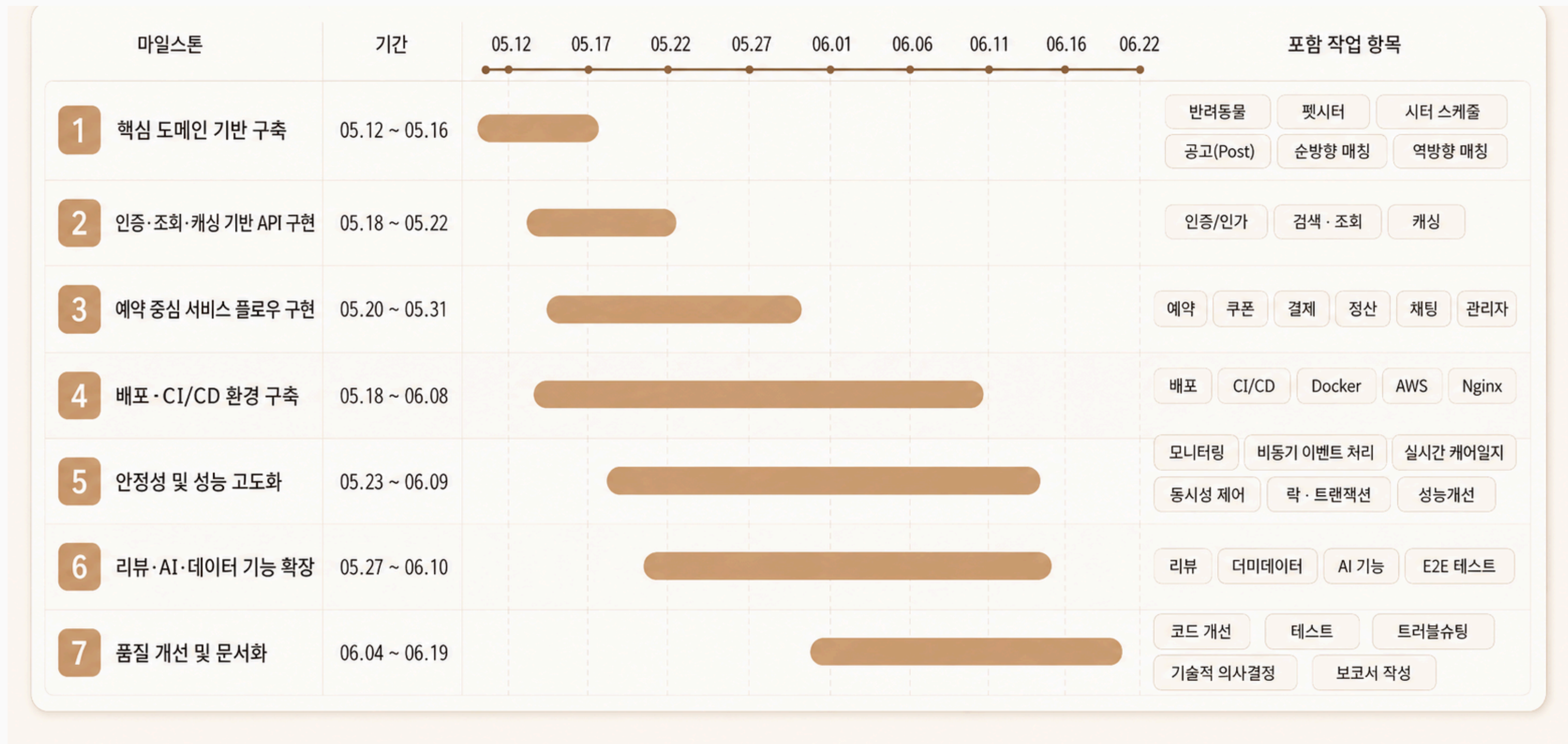
결제·정산 / AI 기능
프론트엔드

4릉흔드..
연두맘





ForPets 마일스톤



설계 및 인프라

ERD

API 설계

협업툴

AWS 아키텍처

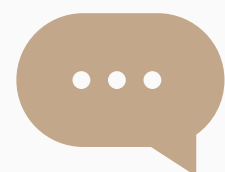
개발 환경

Git Workflow

Flowchart



디렉토리 구조



3 Layer architecture

도메인 중심 패키지 구조 → 응집도
명확한 레이어별 책임 분리

```

└─ monitoring/
  │ └─ grafana/dashboards/      # Grafana 대시보드 JSON
  │ └─ prometheus*.yaml        # Prometheus 환경별 설정
  │
└─ nginx/                      # 환경별 Nginx 설정
└─ k6/                         # 부하 테스트 스크립트
└─ Dockerfile
└─ docker-compose.yml
└─ docker-compose.prod.yml
└─ build.gradle
  
```

```

for-pets-project/
└─ src/main/java/com/forpets/
  │ └─ domain/
  │ │ └─ admin/                # 관리자 (예약·시터 관리)
  │ │ └─ ai/                   # AI (채팅·RAG 벡터 검색·리뷰 요약·사용량 로그)
  │ │ └─ auth/                 # 인증 (JWT 발급·재발급)
  │ │ └─ carelog/              # 케어 일지
  │ │ └─ carerequest/          # 케어 요청
  │ │ └─ chat/                 # 실시간 채팅 (WebSocket + Redis Stream)
  │ │ └─ coupon/               # 쿠폰 발급·사용
  │ │ └─ member/               # 회원
  │ │ └─ notification/         # 알림 (SSE + Kafka)
  │ │ └─ payment/              # 결제 (토스페이먼츠)
  │ │ └─ pet/                  # 반려동물 등록·관리
  │ │ └─ post/                 # 구인·구직 게시글
  │ │ └─ proposal/             # 시터 제안
  │ │ └─ reservation/         # 예약
  │ │ └─ review/               # 리뷰
  │ │ └─ settlement/           # 정산
  │ │ └─ sitter/               # 시터 프로필
  │ │
  │ └─ global/
  │ │ └─ config/               # Redis·Kafka·WebSocket·QueryDSL 설정
  │ │ └─ exception/           # 공통 예외 처리
  │ │ └─ security/             # Spring Security, JWT
  │ │ └─ scheduler/            # 만료 처리 스케줄러
  │ │ └─ aspect/               # 분산 락 AOP
  │ │ └─ monitoring/           # 실행 시간 추적 AOP
  │ │ └─ sse/                  # SSE 연결 관리
  
```


✓ API 명세서

☰ 실시간 커머스 Spring 3기 / 주차별 팀노션 / 집사조일체						
이 페이지는 링크가 있는 모든 사용자에게 표시됩니다. ⓘ						
⚙️ 명세 작성 현황	📁 기능 분류	Aa 기능명	≡ API Path	📄 http me...	👤 담당자	☑️ 테스트 여부
● 완료	인증	📄 회원 탈퇴	/api/members/me	DELETE	최길중	☑️
● 완료	관리자	📄 승인 대기 상태 시터 목록 조회	/api/admin/sitters	GET	권지원	☑️
● 완료	관리자	📄 시터 승인	/api/admin/sitters/{sitterProfileId}/approve	POST	권지원	☑️
● 완료	관리자	📄 시터 거절	/api/admin/sitters/{sitterProfileId}/reject	POST	권지원	☑️
● 완료	관리자	📄 불가피한 취소 요청 목록 조회	/api/admin/reservations/cancel-requests	GET	권지원	☑️
● 완료	관리자	📄 취소 승인	/api/admin/reservations/{reservationId}/cancel-approve	POST	권지원	☑️
● 완료	관리자	📄 취소 거절	/api/admin/reservations/{reservationId}/cancel-reject	POST	권지원	☑️
● 완료	반려동물	📄 반려동물 등록	/api/pets	POST	권지원	☑️
● 완료	반려동물	📄 내 반려동물 목록 조회	/api/pets	GET	권지원	☑️
● 완료	반려동물	📄 반려동물 상세 조회	/api/pets/{petId}	GET	권지원	☑️
● 완료	반려동물	📄 반려동물 정보 수정	/api/pets/{petId}	PUT	권지원	☑️
● 완료	반려동물	📄 반려동물 삭제	/api/pets/{petId}	DELETE	권지원	☑️
● 완료	시터	📄 시터 프로필 등록	/api/sitters	POST	권지원	☑️
● 완료	시터	📄 시터 프로필 조회	/api/sitters/me	GET	권지원	☑️
● 완료	시터	📄 시터 프로필 수정	/api/sitters/me	PUT	권지원	☑️
● 완료	시터	📄 시터 프로필 상태 전환	/api/sitters/me/status	PUT	권지원	☑️
● 완료	시터	📄 시터 삭제 (탈퇴)	/api/sitters/me	DELETE	권지원	☑️
● 완료	시터	📄 시터 목록 검색 (동적 필터)	/api/sitters	GET	선경안	☑️
● 완료	시터	📄 시터 상세 조회	/api/sitters/{sitterId}	GET	선경안	☑️
● 완료	시터 스케줄	📄 스케줄 등록/수정/삭제	/api/sitters/me/schedules	PUT	권지원	☑️
● 완료	시터 스케줄	📄 내 스케줄 조회	/api/sitters/me/schedules	GET	권지원	☑️

☰

실시간 커머스 Spring 3기 / 주차별 팀노션 / 집사조일체

☑️

기능 분류

시터

≡

API Path

/api/sitters/me

📄

http method

PUT

👤

담당자

권지원

⚙️

명세 작성 현황

● 완료

☑️

테스트 여부

☑️

+

Add a property

💬

댓글

🌐 댓글 추가

Description

내 시터 프로필을 수정한다.

필수 값 누락 또는 형식 오류: 시간당 요금, 경력 연수가 음수 존재하지 않는 Enum 값 입력 불가능

등록이 거절 또는 대기 상태인 경우 불가능

Header

Authorization : Bearer {accessToken}

Request Body

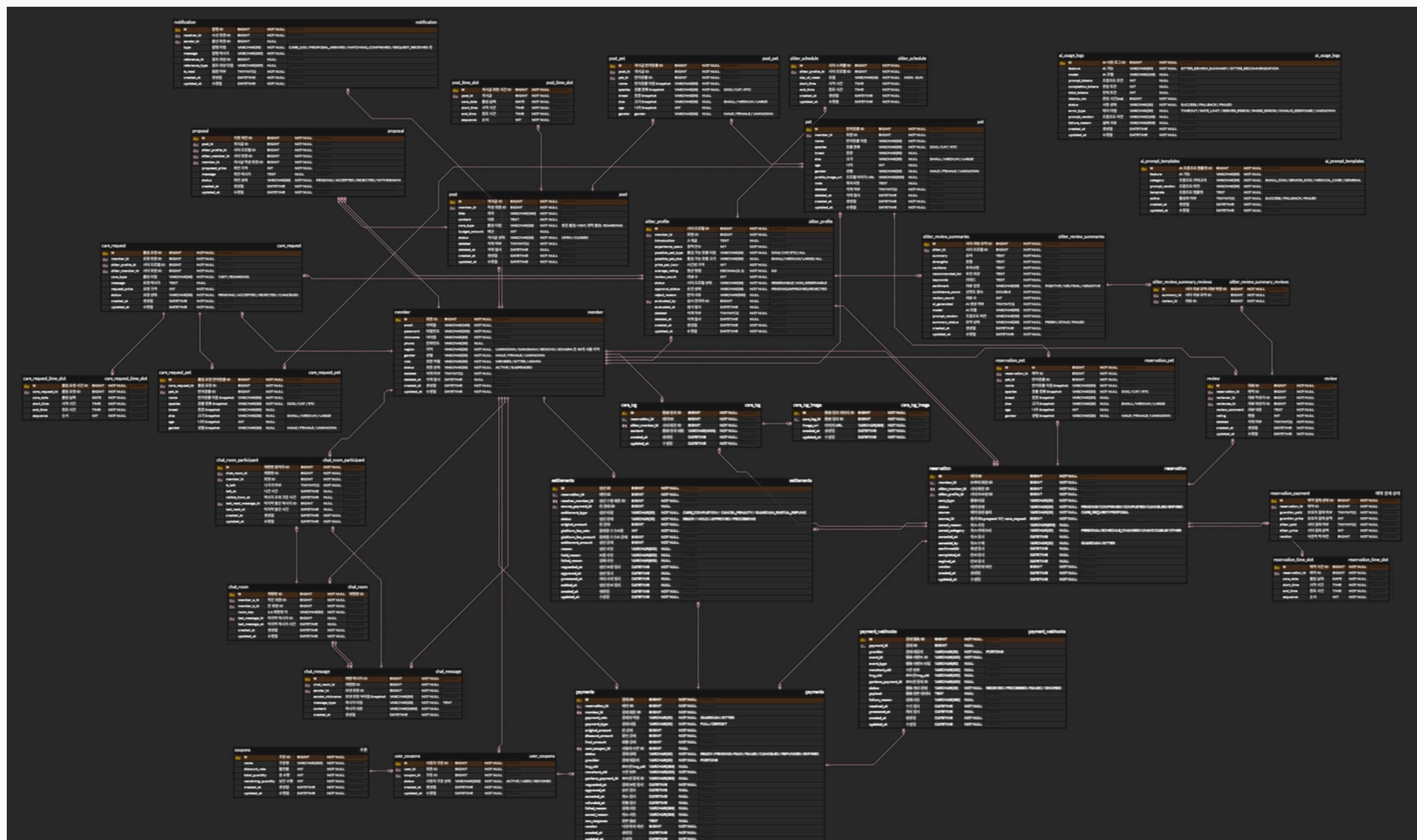
{

"introduction": "이제 거대 고양이도 가능하니까 시급을 올렸습니다~",

"experienceYears": 10,

}

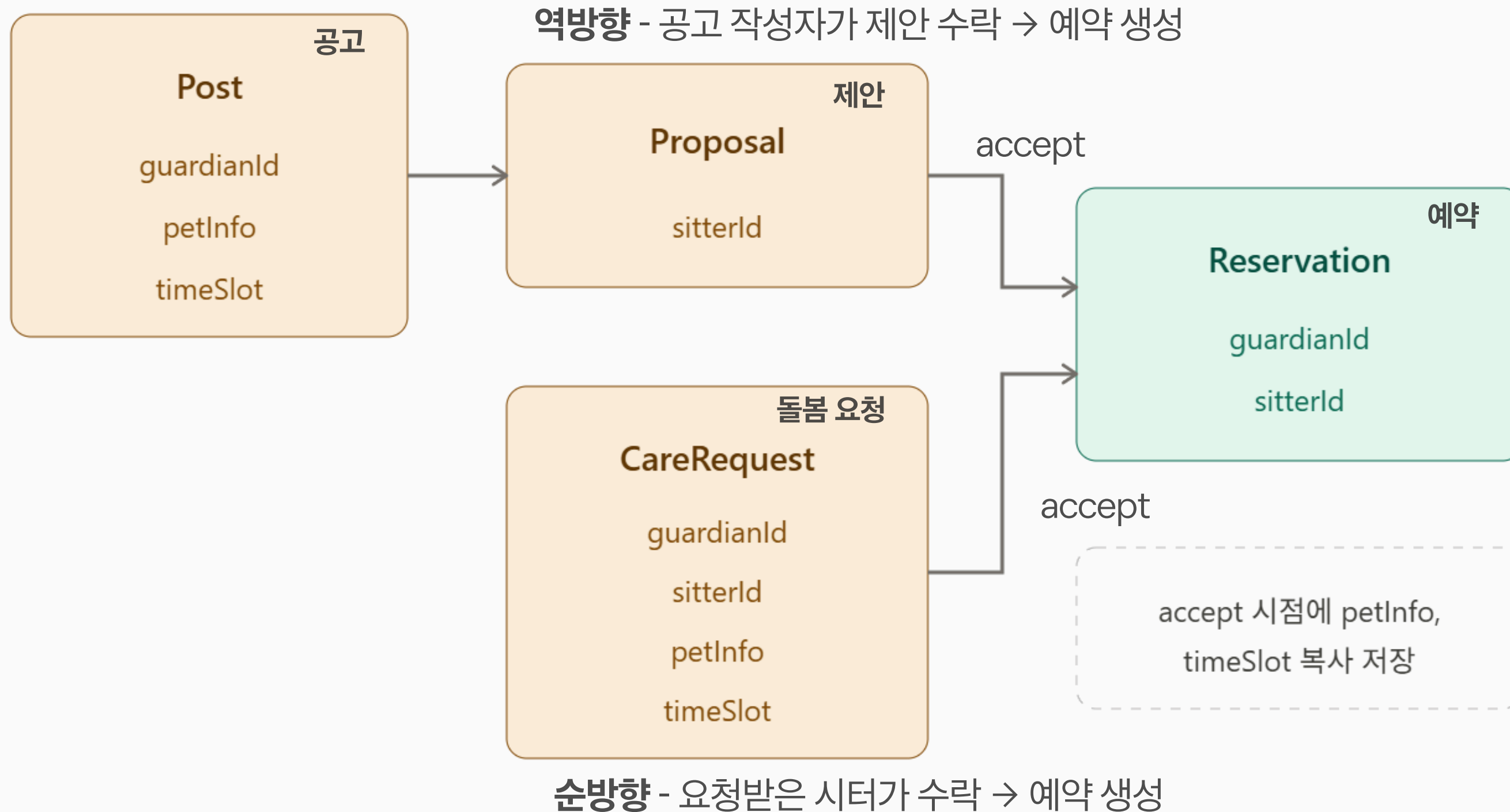
ERD





ERD

매칭 생성 부분 ERD 흐름



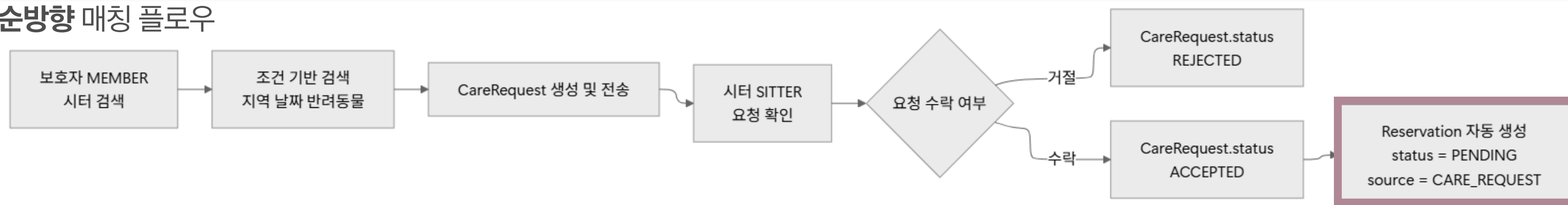


Flowchart

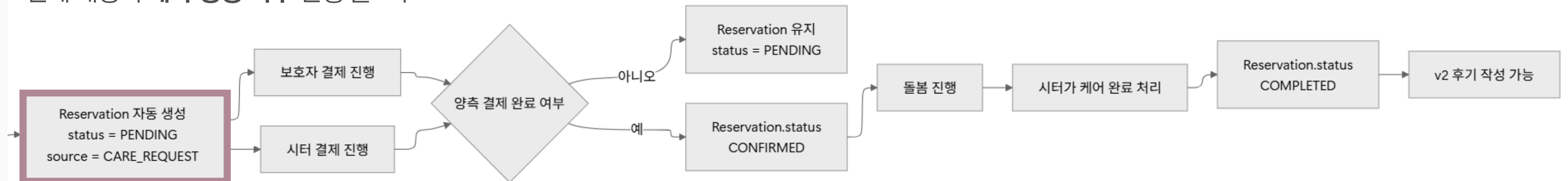
역방향 매칭 플로우



순방향 매칭 플로우



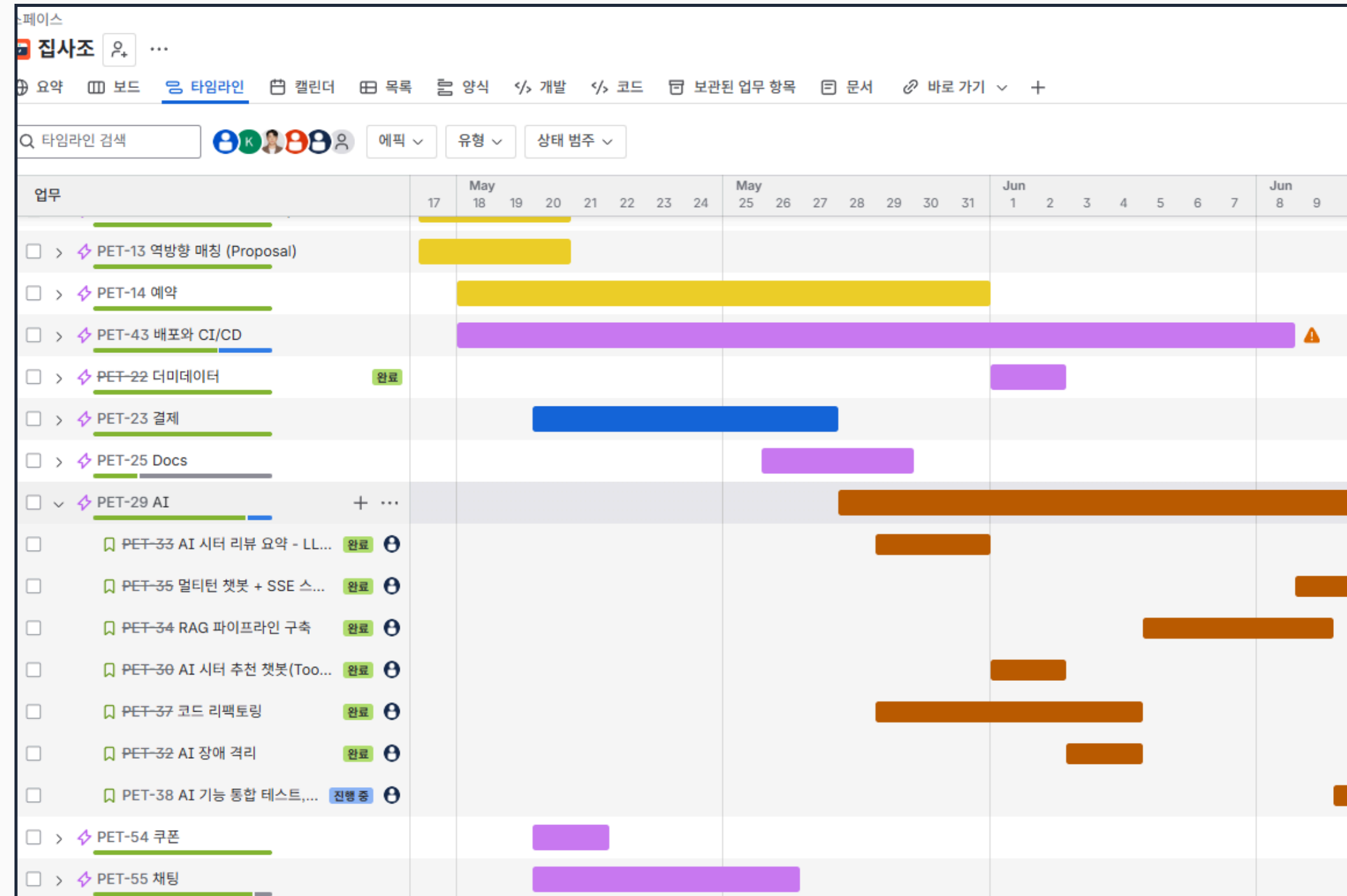
전체 매칭의 예약 생성 이후 진행 플로우





기술 스택

Backend	Java 21Spring Boot 3.5.14Spring WebSpring Data JPASpring SecurityQueryDSLJWTLombok
Database	MySQL 8.4H2
Vector DB	Qdrant 1.9.7
Cache / Lock	Redis 7Redisson 3.27.2
Messaging	Kafka 3.7.0
Real-time	WebSocketSTOMP
AI	Spring AIGemini API
Infra / DevOps	DockerDocker ComposeNginxAWS EC2AWS ECRGitHub Actions
Test / Monitoring	JUnit 5k6PrometheusGrafanaActuatorMicrometer
Collaboration	GitHubNotionIntelliJ IDEAPostman





협업툴

문서 - Notion

2. JWT / 인증 정책

- Access Token 만료 시간은 1시간이다.
- 로그아웃 시 Access Token을 Redis 블랙리스트에 등록한다.
키: jwt:blacklist:{token} / TTL: 토큰 잔여 만료 시간\ Refresh Token도 함께 무효화한다.
- 블랙리스트에 등록된 토큰은 JwtAuthFilter에서 차단한다.
- Refresh Token 만료 시간은 7일
- Refresh Token 저장 위치 : Redis
- Refresh Token 재발급 시 기존 토큰을 무효화한다. (Rotation 전략)

지원 권 5월 15일
길중님 구현 추가 파트

3. 시터 프로필 정책

- MEMBER 가 시터 프로필을 작성하여 등록을 요청하면 ADMIN 이 approve api 를 통해 member.role = SITTER로 변경한다.
- 시터 프로필 삭제 시 member.role = MEMBER로 복원된다.
- 시터 프로필 재등록 시 다시 ADMIN 승인이 필요하고, 받은 리뷰 등은 복구된다.
- ADMIN은 시터 프로필을 등록할 수 없다. -> DB 에 직접 등록, 모든 권한 부여
- 회원 1명은 시터 프로필을 1개만 등록할 수 있다.
- 진행 중 예약이 있으면 시터 프로필을 삭제할 수 없다.
- 시터의 상태를 RESERVABLE / NON_RESERVABLE 로 구분
- 상태 전환:
 - 시터가 직접 토글 가능
 - 진행 중 예약(PENDING/CONFIRMED) 있어도 NON_RESERVABLE로 변경 가능
 - NON_RESERVABLE 상태에서는 신규 CareRequest/Proposal 불가

지원 권 5월 15일
삭제 이후 UI 에서 검색 관련한 정책을 다시 정리

지원 권 5월 15일
Member 쪽으로 정책 이동하거나 MVP 기준으로는 ADMIN 을 완전 제외하여 Sitter profile 을 등록하려고 하면 바로 Sitter 역할을 부여

4. 시터 주간 스케줄 정책

시터 스케줄 역할

공지 및 PR 알림 - Slack

박영수(Spring_3기) 오후 3:21
@here 금일 한 16시 쯤에 main배포 예정입니다.(nginx적용, 도메인 DNS 연결)
경안님처럼 prod 에 db 테이블 변경이 필요한 경우 저한테 쿼리랑 같이해서 내용 공유주시면 RDS에 반영 (그전에 PR 있으신분들은 미리끝내주세요! 아니면 배포이후에 부탁드립니다.)

cc. @선경안(Spring_3기) (편집됨)

넵! 3 🤔

GitHub 앱 오후 3:25
Pull request opened by giljung8228

#128 Feature/pet 215

- 쿠폰 단위, 통합 테스트 코드
- 인증 단위, 통합 테스트 코드
- 채팅 단위, 통합 테스트 코드

Reviewers

@박영수(Spring_3기)

Housekeeper-for-pets/for-pets-project | 6월 9일 | 봇이 추가한 GitHub

Comment

1개의 댓글 7일 전

GitHub 앱 오후 3:26
Pull request opened by jiiimni

#129 Feature/pet 34

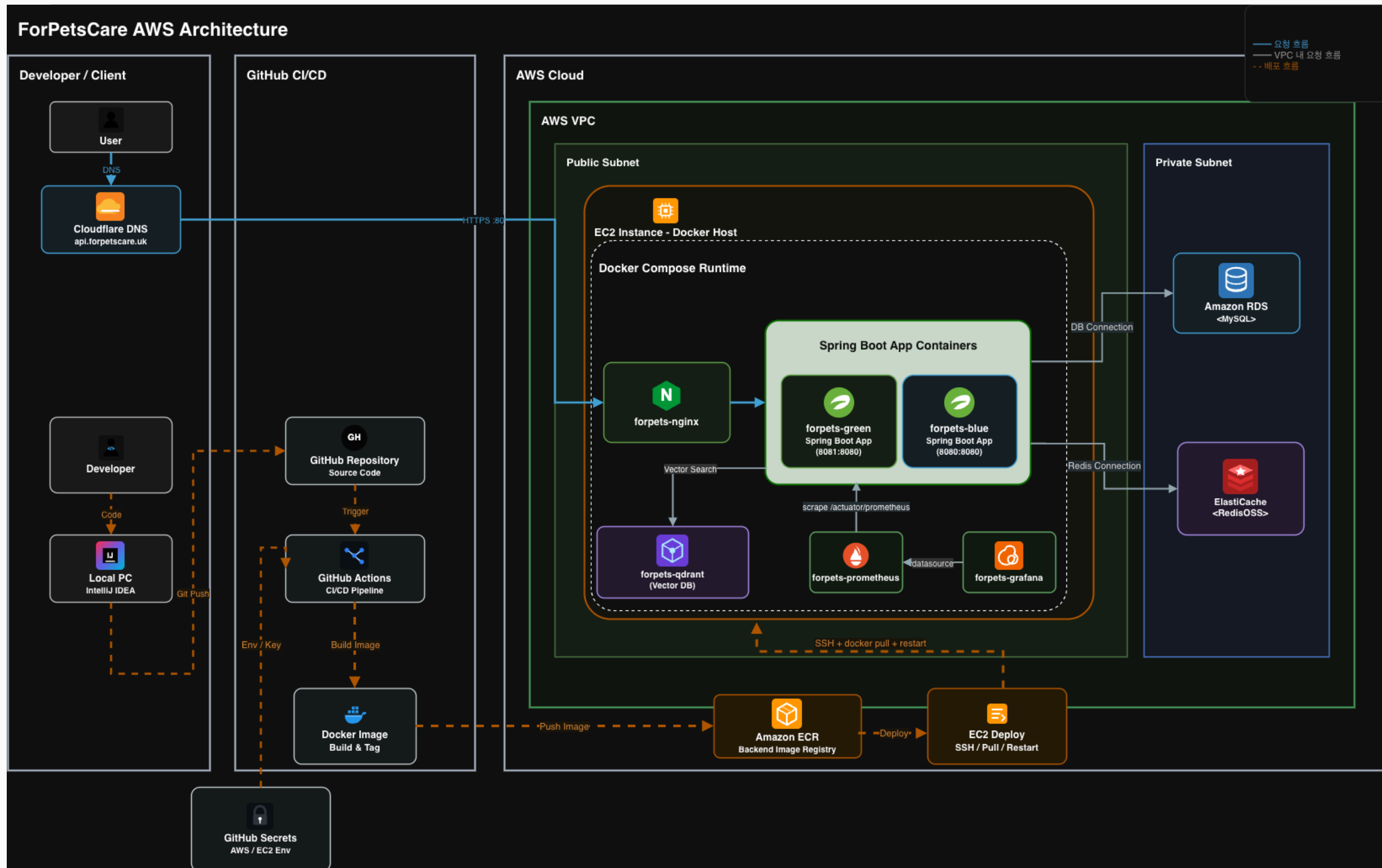
Reviewers

@박영수(Spring_3기), @grown-tree, @권지 원(Spring_3기)

Assignees

@jiiimni

✓ AWS 운영 아키텍처



① 요청 진입

User → Cloudflare DNS → Nginx

② 애플리케이션 실행

EC2 내부 Docker Compose 기반
Spring Boot 컨테이너 운영

③ 데이터 계층

RDS(MySQL) / ElastiCache Redis를
Private Subnet에 분리

④ 배포 흐름

GitHub Actions → ECR → EC2 배포
자동화



테스트 결과

IntelliJ 내부 단위테스트, 통합테스트

Project Structure (Left Panel):

- auth
- carerequest
- chat
- coupon
- member.service
- payment
- pet
- post
- proposal
- reservation
 - dto
 - ReservationRequestValidationTest
 - service
 - LockTransactionRaceConditionTest
 - OptimisticLockFallbackTest
 - ReservationExpireServiceTest
 - ReservationLockIntegrationTest
 - ReservationLockServiceTest
 - ReservationServiceTest
 - TransactionConnectionWasteTest
- review
- settlement
- sitter
- global
 - cache
 - support
 - GracefulDegradationCacheErrorHandlerTest
 - MeteredCacheTest
- config
- ForPetsApplicationTests

Code Editor (Right Panel):

```
38      -> StaleObjectStateException -> Spring OJ ObjectOptimi
39      */
40      @SpringBootTest  & kwonjiwon
41      @ActiveProfiles("test")
42      class OptimisticLockFallbackTest {
43
44          @Autowired private OptimisticLockProbe probe;
45          @Autowired private ReservationRepository reservationRepository;
46
47          private Long reservationId;  4개 사용 위치
48
49          @BeforeEach  & kwonjiwon
50          void setup() {
51              Reservation r = Reservation.builder()
52                  .guardianId(1L)
53                  .sitterMemberId(2L)
54                  .sitterProfileId(100L)
55                  .careType(CareType.VISIT)
56                  .source(ReservationSource.CARE_REQUEST)
57                  .sourceId(200L)
58                  .build();
59              reservationId = reservationRepository.save(r).getId();
60          }
61
62          @AfterEach  & kwonjiwon
63          void cleanup() { reservationRepository.deleteAll(); }
64
65          @Test  & kwonjiwon
66          @DisplayName("[FALLBACK] 동시에 confirm 시도 - @Version 이 낮은 쪽을")
67          void optimistic_lock_blocks_second_writer() throws Exception {
68              // given
69              CyclicBarrier bothRead = new CyclicBarrier( parties: 2); // 두
70              CountDownLatch aCommitted = new CountDownLatch(1); // A 의 c
```

Postman 기능 단위 시나리오 테스트

A	B	C
번호	분류	테스트 케이스
1	회원/인증	[성공] 유효한 이메일, 비밀번호, 닉네임 입력 시 회원가입 성공 (기본 역할 MEMBER)
2	회원/인증	[실패] 이미 가입된 이메일로 회원가입 시도 시 중복 가입 차단
3	회원/인증	[실패] 이미 사용 중인 닉네임으로 회원가입 시 차단
4	회원/인증	[실패] 비밀번호가 형식 정책을 만족하지 않는 경우 가입 실패
5	회원/인증	[실패] 비밀번호가 형식 정책을 만족하지 않는 경우 가입 실패
6	회원/인증	[성공] 가입된 이메일과 올바른 비밀번호로 로그인 성공 — Access Token, Refresh Token 발급
7	회원/인증	[실패] 존재하지 않는 이메일로 로그인 시도 시 인증 실패
8	회원/인증	[실패] 비밀번호가 일치하지 않을 경우 로그인 실패
9	회원/인증	[실패] 정지된 계정으로 로그인 시도 시 차단
10	회원/인증	[성공] 유효한 Access Token으로 로그아웃 성공 — Redis 블랙리스트 등록
11	회원/인증	[실패] Authorization 헤더 없이 로그아웃 요청 시 차단
12	회원/인증	[실패] 만료된 Access Token으로 로그아웃 요청 시 차단
13	회원/인증	[성공] Refresh Token으로 Access Token 재발급 성공
14	회원/인증	[실패] 만료된 Refresh Token으로 재발급 시도 시 차단
15	회원/인증	[성공] 유효한 Access Token으로 내 정보 조회 성공
16	회원/인증	[성공] 닉네임, 전화번호, 성별 수정 성공
17	회원/인증	[성공] 현재 비밀번호 확인 후 새 비밀번호 변경 성공
18	회원/인증	[실패] 현재 비밀번호 불일치 시 변경 실패
19	회원/인증	[성공] 진행 중 예약 없을 때 회원 탈퇴 성공 — status DELETED 변경
20	회원/인증	[실패] 진행 중 예약이 있을 때 탈퇴 시도 시 차단
21	반려동물	[성공] 로그인된 회원이 반려동물 등록 성공 (name, species 필수)
22	반려동물	[실패] 반려동물 이름 누락 시 등록 실패
23	반려동물	[실패] 허용되지 않는 종 입력 시 등록 실패 (BIRD 등)
24	반려동물	[실패] 반려동물 10마리 초과 등록 시 차단
25	반려동물	[실패] 이름 50자 초과 등록
26	반려동물	[실패] 나이 음수 입력
27	반려동물	[실패] 나이 100 초과 입력
28	반려동물	[실패] 메모 2000자 초과 입력
29	반려동물	[성공] 내 반려동물 목록 조회 성공 — 본인 소유 목록만 반환
30	반려동물	[성공] 반려동물 상세 조회 성공
31	반려동물	[실패] 타인의 반려동물 조회 시 차단
32	반려동물	[실패] 존재하지 않는 반려동물 조회
33	반려동물	[성공] 반려동물 정보 수정 성공
34	반려동물	[실패] 타인의 반려동물 수정 시 차단
35	반려동물	[실패] 존재하지 않는 반려동물 수정
36	반려동물	[성공] 반려동물 삭제 성공 — 연관된 Post/CareRequest/Reservation 없는 경우
37	반려동물	[실패] 타인의 반려동물 삭제
38	반려동물	[실패] 존재하지 않는 반려동물 삭제
39	반려동물	[실패] 연관된 Post/CareRequest/Reservation이 존재하는 반려동물 삭제 시 차단
40	시터 프로필	[성공] MEMBER가 시터 프로필 등록 성공
41	시터 프로필	[실패] experienceYears 누락
42	시터 프로필	[실패] possiblePetType 누락
43	시터 프로필	[실패] possiblePetSize 누락
44	시터 프로필	[실패] pricePerHour 누락
45	시터 프로필	[실패] pricePerHour 0 입력
46	시터 프로필	[실패] pricePerHour 음수 입력



테스트 결과

배포 도메인 E2E 테스트

실시간 커머스 Spring 3기 / ... / E2E 테스트 / For Pets (포펫) - E2E 테스트

Aa TC ID	테스트 케이스명	유형	테스트 여부	배포 테스트	+	...
TC-PET-007	신규 가입 계정으로 로그인 (반려동물 없음)	앳지	Pass	Pass		
+ 새 페이지						

시터 프로필/스케줄 (SITTER)

Aa TC ID	테스트 케이스명	유형	테스트 여부	배포 테스트	+	...
TC-STR-001	시터 프로필 등록	해피패스	Pass	Pass		
TC-STR-002	시터 목록 검색 (비로그인)	해피패스	Pass	Pass		
TC-STR-003	시터 목록 검색 - 지역 필터	해피패스	Pass	Pass		
TC-STR-004	시터 상세 조회	해피패스	Pass	Pass		
TC-STR-005	내 시터 프로필 조회	해피패스	Pass	Pass		
TC-STR-006	시터 프로필 수정	해피패스	Pass	Pass		
TC-STR-007	예약 상태 변경 (예약가능 ↔ 불가)	해피패스	Pass	Pass		
TC-STR-008	시터 프로필 삭제	해피패스	Pass	Pass		
TC-STR-009	시터 스케줄 등록	해피패스	Pass	Pass		
TC-STR-010	시터 스케줄 조회	해피패스	Pass	Pass		
TC-STR-011	시터 프로필 없는 회원의 시터 기능 접근	예외	Pass	Pass		
+ 새 페이지						

돌봄 요청 (CARE REQUEST)

Aa TC ID	테스트 케이스명	유형	테스트 여부	배포 테스트	+	...
TC-CRQ-001	시터에게 돌봄 요청 전송	해피패스	Pass	Pass		
TC-CRQ-002	보낸 돌봄 요청 목록 조회	해피패스	Pass	Pass		
TC-CRQ-003	돌봄 요청 상세 조회	해피패스	Pass	Pass		

4. 변경 확인

5. 다시 예약가능으로 재변경 후 확인

기대 결과

상태 즉시 반영
시터 조회 시 예약 불가능 상태라고 출력

실제 결과

기대 결과와 동일

캡처, 비교

ForPets | 포펫츠

localhost:5173/my-sitter?mode=summary

로그아웃

공고 검색

MY SITTER PROFILE

내 시터 프로필

시터 프로필을 등록하고 예약 가능 시간과 상태를 관리합니다.

승인됨

예약 가능 상태가 변경되었습니다.

SUMMARY

시터 프로필 요약

상세보기

STATUS

예약 가능 상태

예약 가능

예약 불가

세부 기능 구현

주요 기능

세부 기능

- 쿠폰 발급 동시성
- 성능 개선 (인덱싱/캐싱/결과)
- 알림 고도화
- AI

시연 영상



쿠폰 발급 - 정합성

평가 기준: 어떤 Lock 이 **초과 발급**을 방지하는가?

시나리오 1

사용자 300명, 쿠폰 100개
 (일반 경합)

시나리오 2

사용자 500명, 쿠폰 10 개
 (극단적 경합)

Optimistic Lock 파라미터

- maxRetry: 10회
- backoffMs: 50ms

전략 \ 시나리오	S-1 (300명/100개)	S-2 (500명/10개)	정합성	소요시간
No 락	300 발급 (200 초과) ❌	324 발급 (314 초과) ❌	❌	1,630ms
낙관적 락	92 발급 (8 부족) ⚠️	10 발급	⚠️	4,085ms
비관적 락	100 발급	10 발급	✅	2,050ms
분산 락	100 발급	10 발급	✅	2,420ms



쿠폰 발급 - 채택: Redis 분산락

Redis 분산락 채택 이유

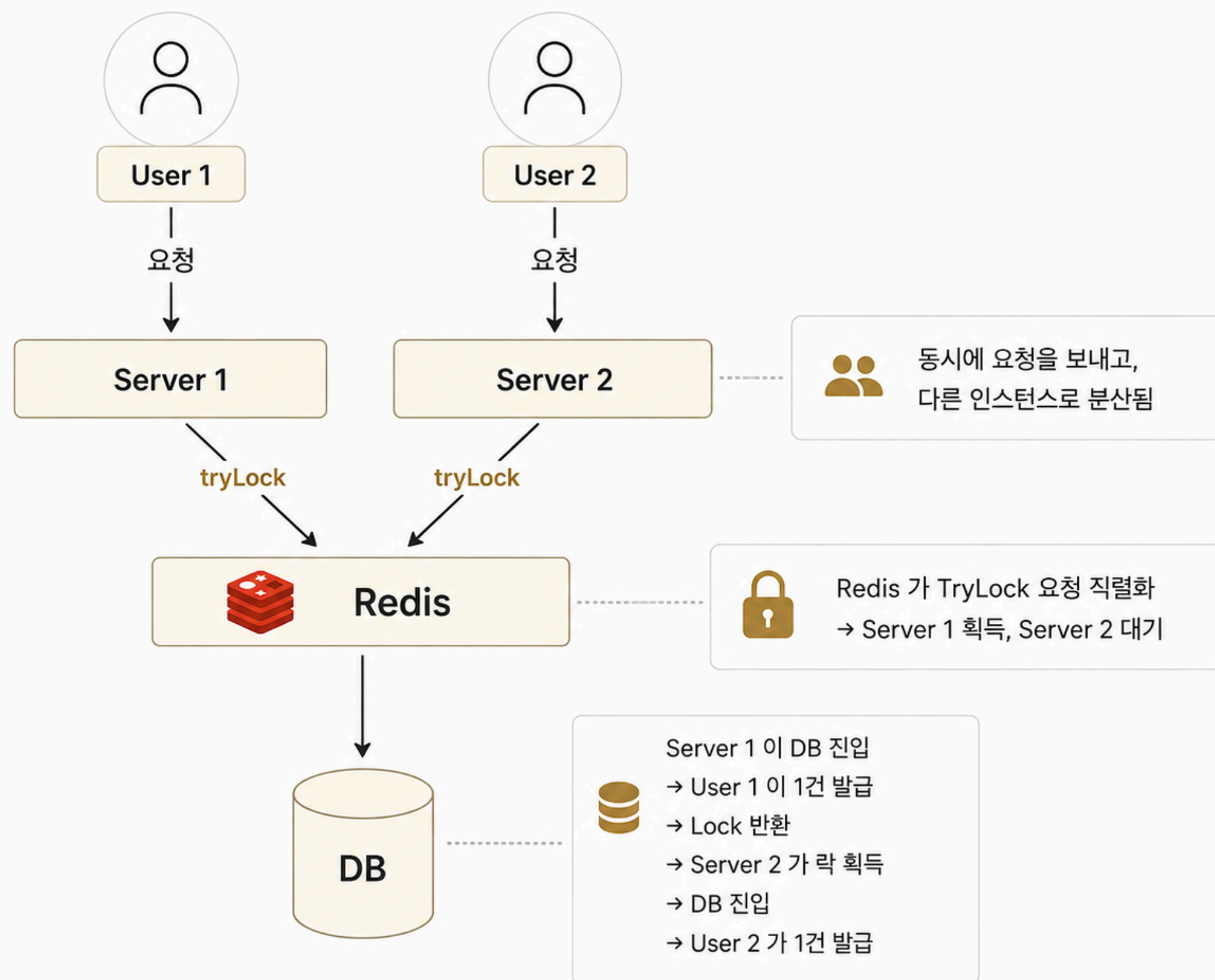
- 1. 다중 인스턴스 환경 대응 가능
- 2. DB 자원 보호
- 3. 인프라 추가 비용 없음

Redis 세팅

락 키: lock:coupon:issue:{couponId}

락 단위: 쿠폰 ID 단위 (도메인 영향 최소화)

실패 응답: 409 COUPON_ISSUE_LOCK_FAILED





쿠폰 발급 - 임계값

임계값 모니터링 테스트

	Pessimistic (비관락)			Distributed (분산락)		
VU	성공	HikariCP pending	500 오류	성공	pending	500 오류
100	100	0	0	100	0	0
300	300	0	0	300	0	0
325	—	—	—	325 ✓	0 (875ms 대기)	0
350	350	119 ⚠ (2.34s 대기)	0	340	0	0
400	291 ✗	165	109 ✗	87	0	0

한계점과 개선방향

Redis 단일 인스턴스 의존

Redis 장애 시 락 자체 동작 불가

개선: Redis Sentinel 검토
 자동 페일오버로 가용성 확보

waitTime 3초

사용자 체감 시간

개선: 큐 기반 비동기 발급
 (Kafka) 으로 전환 검토

락 실패(409)의 클라이언트 처리 정책

자동 재시도 정책

개선: 재시도 정책과 더불어
 대기열 UI 도입



성능개선 개요

성능 개선 대상 쿼리

GET /api/sitters
GET /api/sitters/{id}
GET /api/posts

구조

동적 where, ordering, Pagination

검색 조건

region · gender · petType ·
petSize · minPrice · maxPrice

더미 데이터 구성

규모

member 10만 + sitter_profile 10만

현실과 유사한 **데이터 분포** 구성

ex) 이용가능 85%, 이용 불가능 15%
승인 75%, 대기 15%, 거절 10% 등

재현성

Random seed 고정

테스트 설정

도구

k6

시나리오

constant-vus: 50 VU * 3m
요청 간격: 100ms
목표 요청 수 30,000 +
p95 < 200ms · p99 < 200ms
error rate < 1%

랜덤으로 검색 조건을 선택하여 요청
→ 실 트래픽 모사

✓ 성능개선 — Index, Cache

Index 도입 이유

동적 조건 쿼리 검색
조건 조합별 인덱스 채택 상이
Member table 과 JOIN 존재

➡ **쿼리를 빠르게**

- (possible_pet_type, created_at) - sitter_profile
- (possible_pet_type, created_at, possible_pet_size, approval_status) - sitter_profile
- (region, gender) - member

선택 기준

등치 조건, 정렬(created_at) 컬럼 우선
Cardinality 고려
EXPLAIN 분석 (eq_ref 사용: **100,003 건 → 4,004 건**)

Cache 도입 이유

시터 프로필: read-heavy 도메인
동일 검색 조건 반복 요청 가능

➡ **DB 를 쉬게**

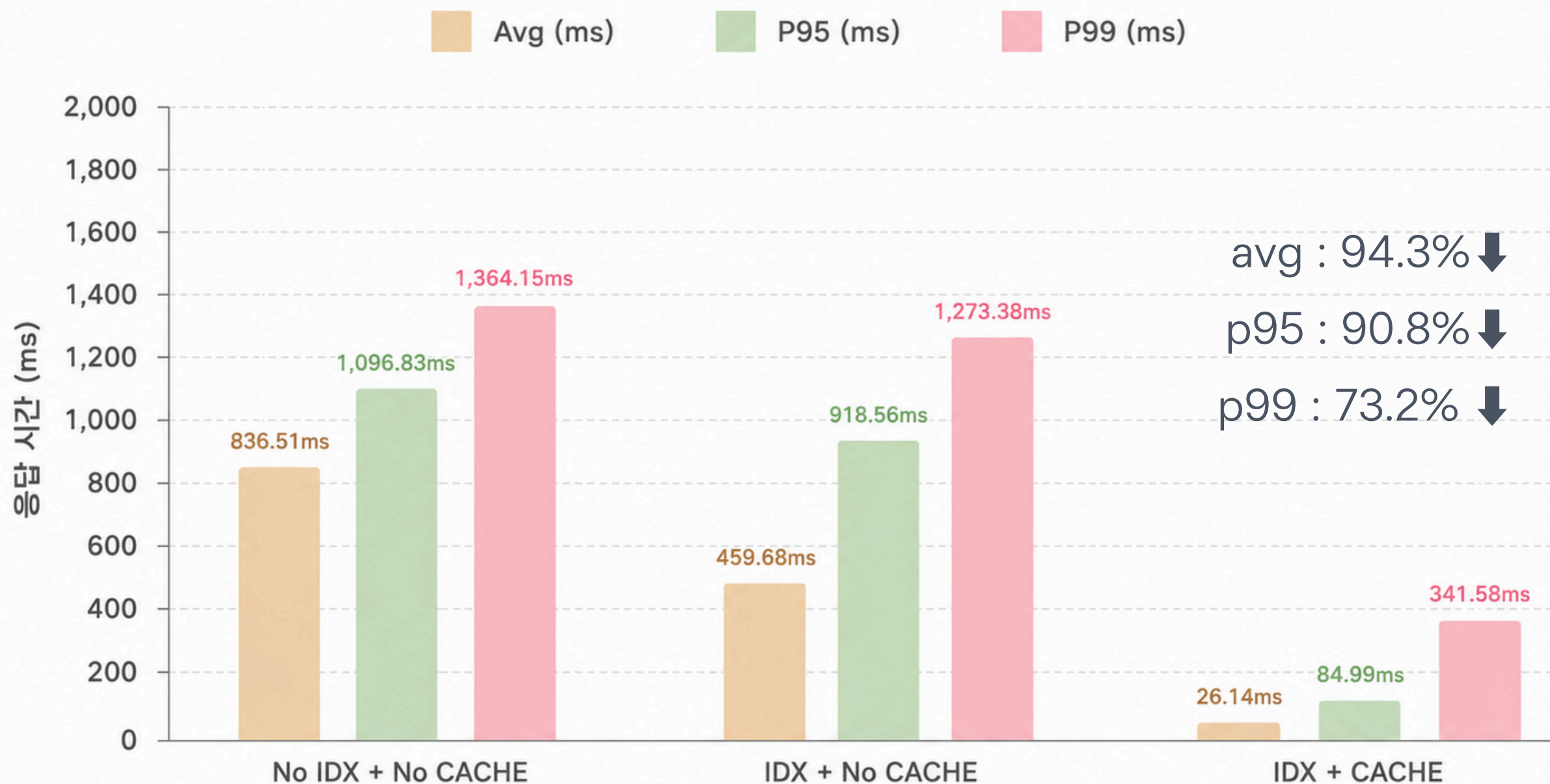
	longTtlCacheManager	shortTtlCacheManager
TTL	1 H	5 m
대상	sitters 목록 sitter 단건	postings 목록

캐시 키 설계 : cache:sitters:{sha-256 hash}

10개 파라미터 해시(검색조건+페이징·정렬) : SHA 256
→ Cache key 길이 일정, 유일성 보장

✓ 성능개선 — 결과

시터 목록 조회 성능 비교



추가 지표

캐시 적중률 **88.5 %**

(전체 요청: 46,554개)

RPS **7.38배**

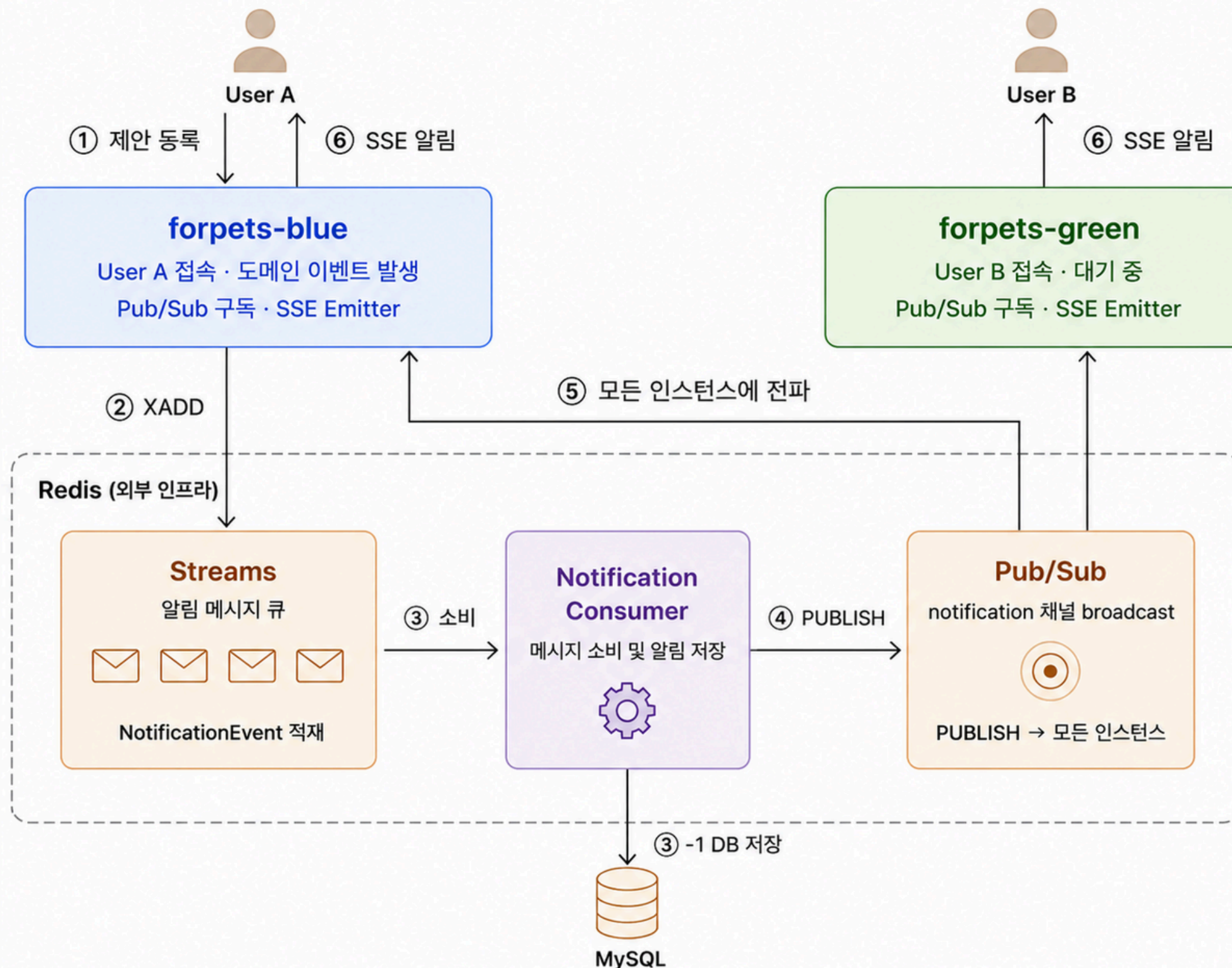
(no IDX + no CACHE vs IDX + CACHE)

53.11 → 392.07

개선 방향

- 1 운영 전 캐시 워밍업 전략
- 2 캐시 무효화 범위 최적화

실시간 알림 시스템 고도화



- ① 도메인 이벤트 발생
- ② Redis Streams 발행
- ③ Consumer 메시지 소비 및 DB 저장
- ④ Redis Pub/Sub 발행
- ⑤ 모든 앱 인스턴스에 전파
- ⑥ 연결된 인스턴스에서 SSE 전송

NOTIFICATIONS

알림

요청, 제안, 결제처럼 바로 확인해야 하는 이벤트를 모아 봅니다.

안 읽은 알림

미읽음 1건

예약 만료 미읽음

관련 화면

읽음 처리

예약 결제 제한 시간이 초과되어 예약이 만료되었습니다.

2026. 6. 19. 오후 11:22

결제 완료 읽음

관련 화면

읽음 완료

결제가 완료되어 예약이 확정되었습니다.

2026. 6. 19. 오후 9:41

✓ DB 저장 후 Pub/Sub 발행
실시간 전송 실패 시에도 알림 이력 보존

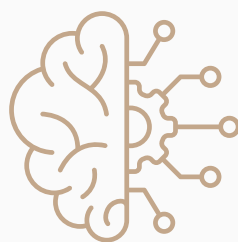
✓ AI 기능 소개

✓ 단순 조건 매칭을 넘어, 시터 정보와 실제 보호자 리뷰 근거를 함께 활용해 맞춤형 시터 추천과 리뷰 요약을 제공



사용자 질문 입력

지역, 일정,
반려동물 성향 등
원하는 돌봄 조건 입력



AI 조건 분석

사용자 요청에서
추천 기준과
핵심 조건 추출



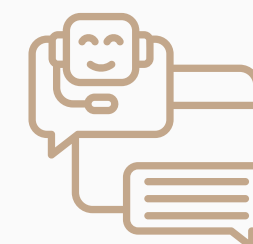
시터 후보 조회

지역, 일정,
서비스 조건에 맞는
펫시터 후보 조회



RAG 리뷰 검색

Qdrant에서
실제 리뷰 기반
유사 근거 데이터 검색



AI 응답 생성

Gemini가
시터 정보와
리뷰 근거를 바탕으로
추천 생성



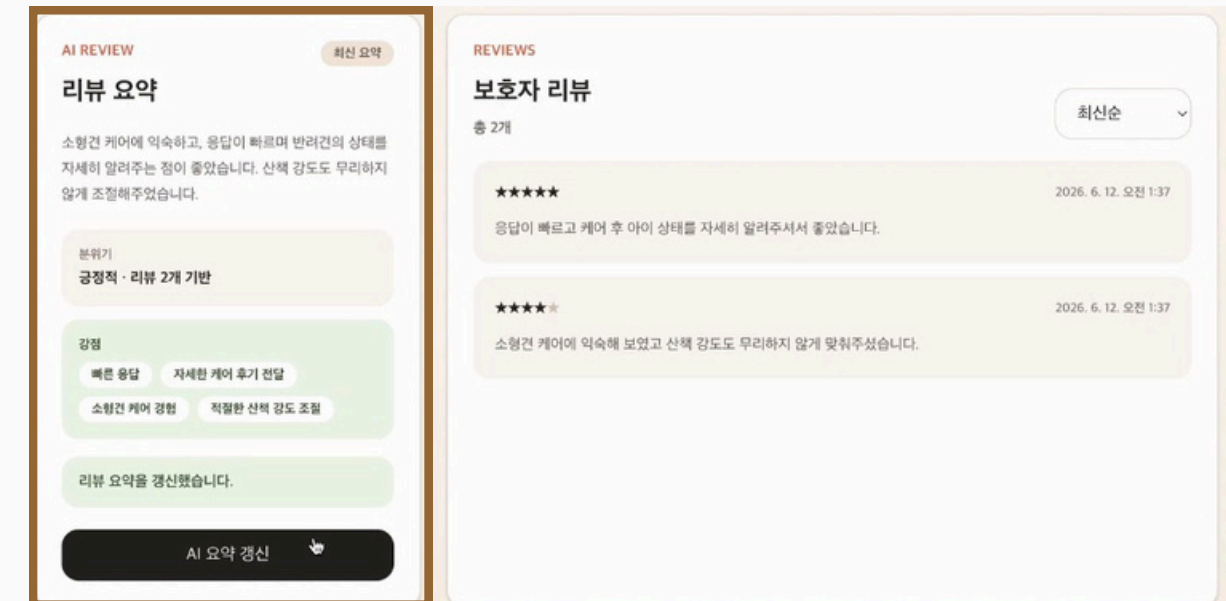
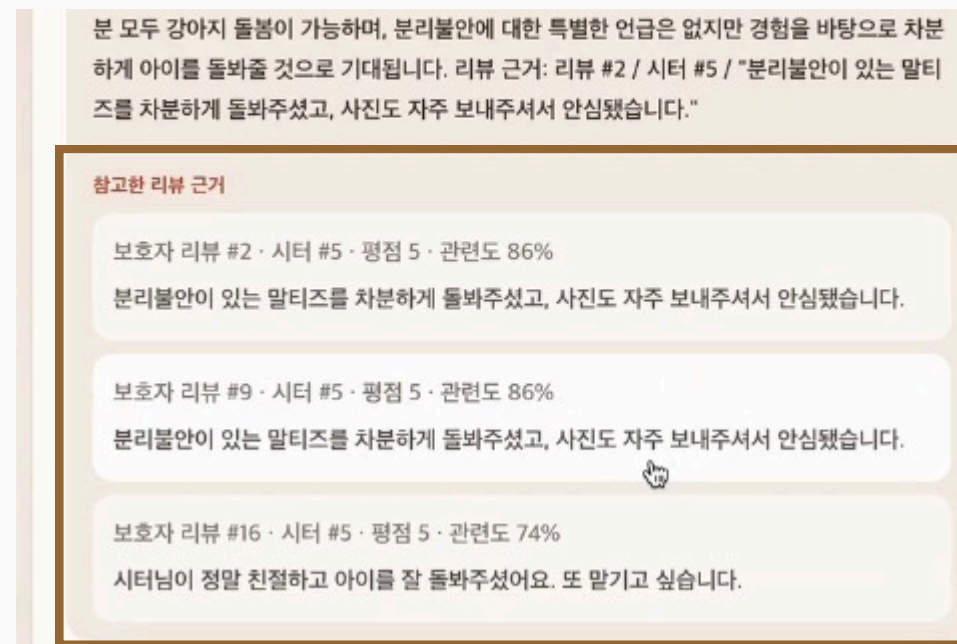
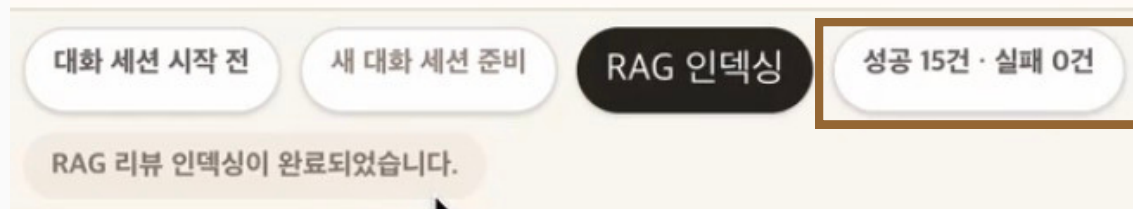
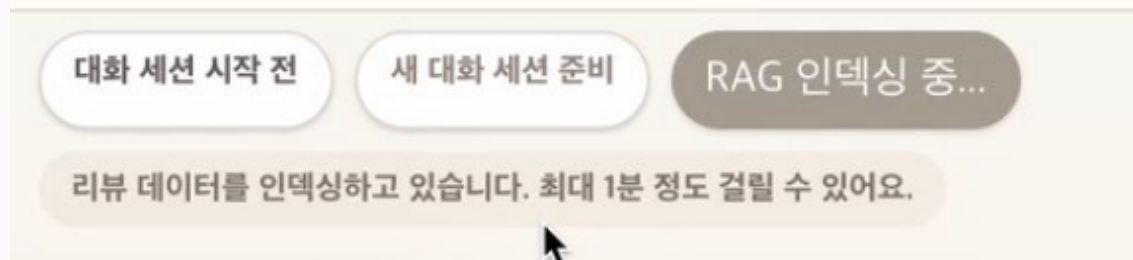
추천 결과 제공

추천 시터 카드와
리뷰 근거를
함께 표시



AI 기능 구현 디테일

관리자 계정으로 접속 시 인덱싱 가능



RAG 인덱싱 관리

리뷰 데이터를 Qdrant에 적재해
AI 추천과 리뷰 요약의 근거로 활용

→ 관리자 전용 인덱싱 기능을 두고
성공/실패 건수를 분리해 추적

RAG 검색 결과 주입 구조

Qdrant에서 조회한 리뷰 데이터를
AI 프롬프트의 근거 정보로 전달

→ 생성된 추천 응답과 함께
사용된 리뷰 근거를 화면에 매핑해 표시

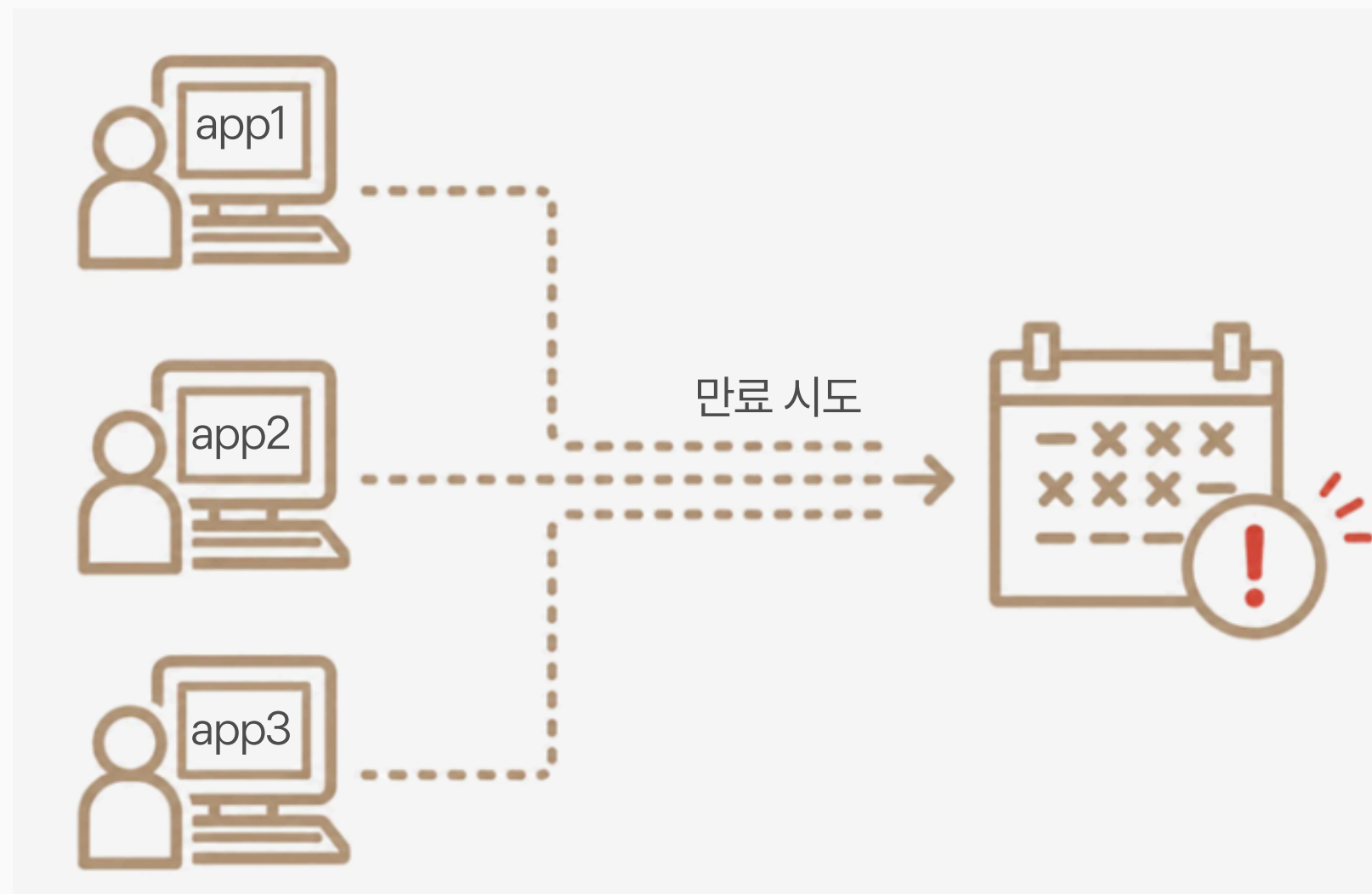
프롬프트 템플릿 관리

배포 환경에서 활성화된 프롬프트 템플릿이
없어 AI 리뷰 요약 생성 실패

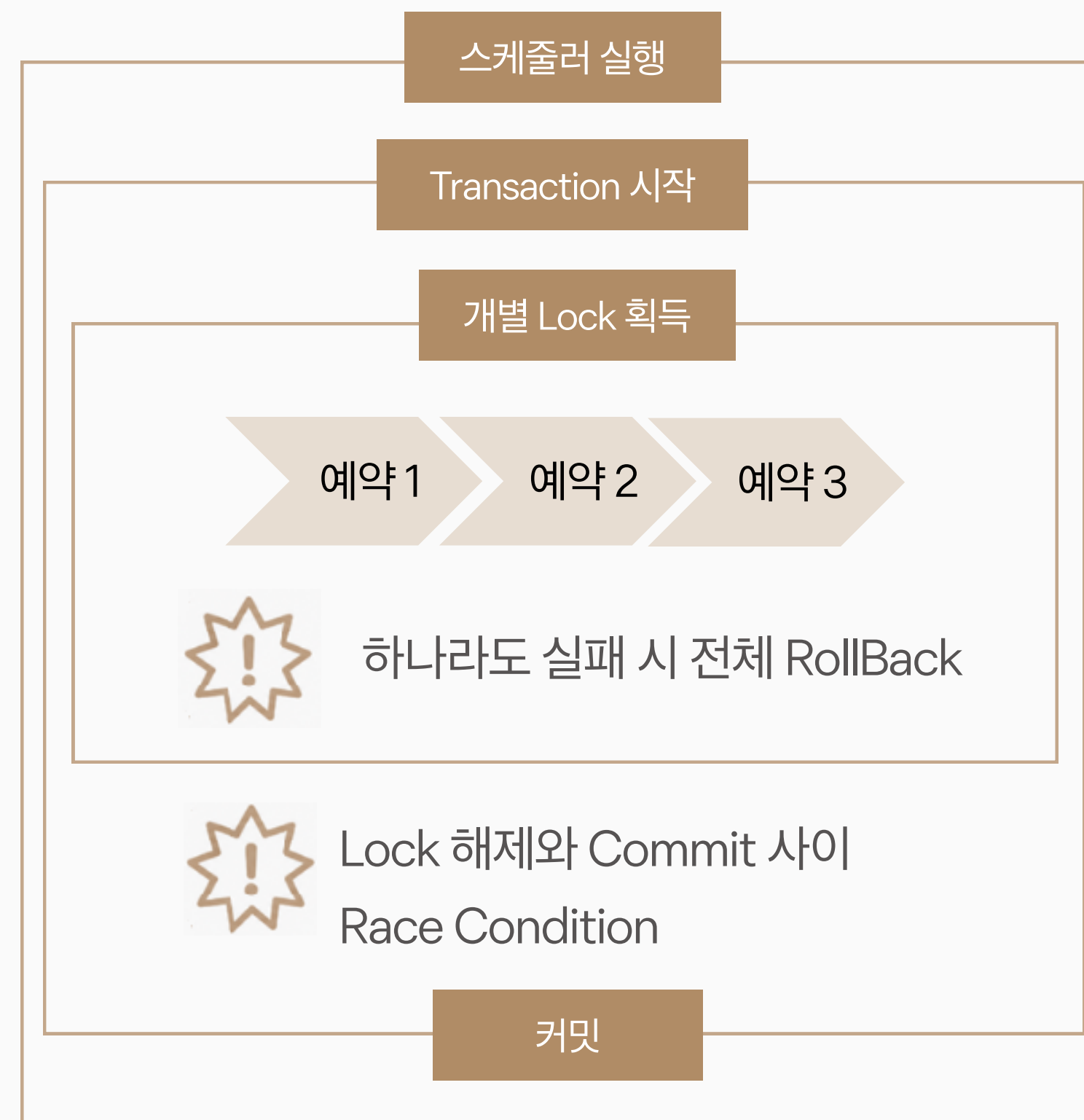
→ 환경별 프롬프트 로딩 구조를 점검하고
배포 환경에서도 리뷰 요약이 생성되도록 수정

✓ 다중 인스턴스 환경 + 동시성 문제

다중 인스턴스 환경 (스케줄러)



자원 하나에 여러 요청 발생 시
 단일 인스턴스 Lock 으로는 대응 불가





리팩토링, 2중 락, ShedLock

리팩토링

목록 단위 Tx → 단건 단위 Tx
부분 실패 격리

스케줄러 실행

개별 Transaction + Lock

예약 1

개별 Transaction + Lock

예약 2

개별 Transaction + Lock

예약 3

분산락

기본 동시 접근 차단

① Lock Service 분리
→ Lock 위치가 TX 외부

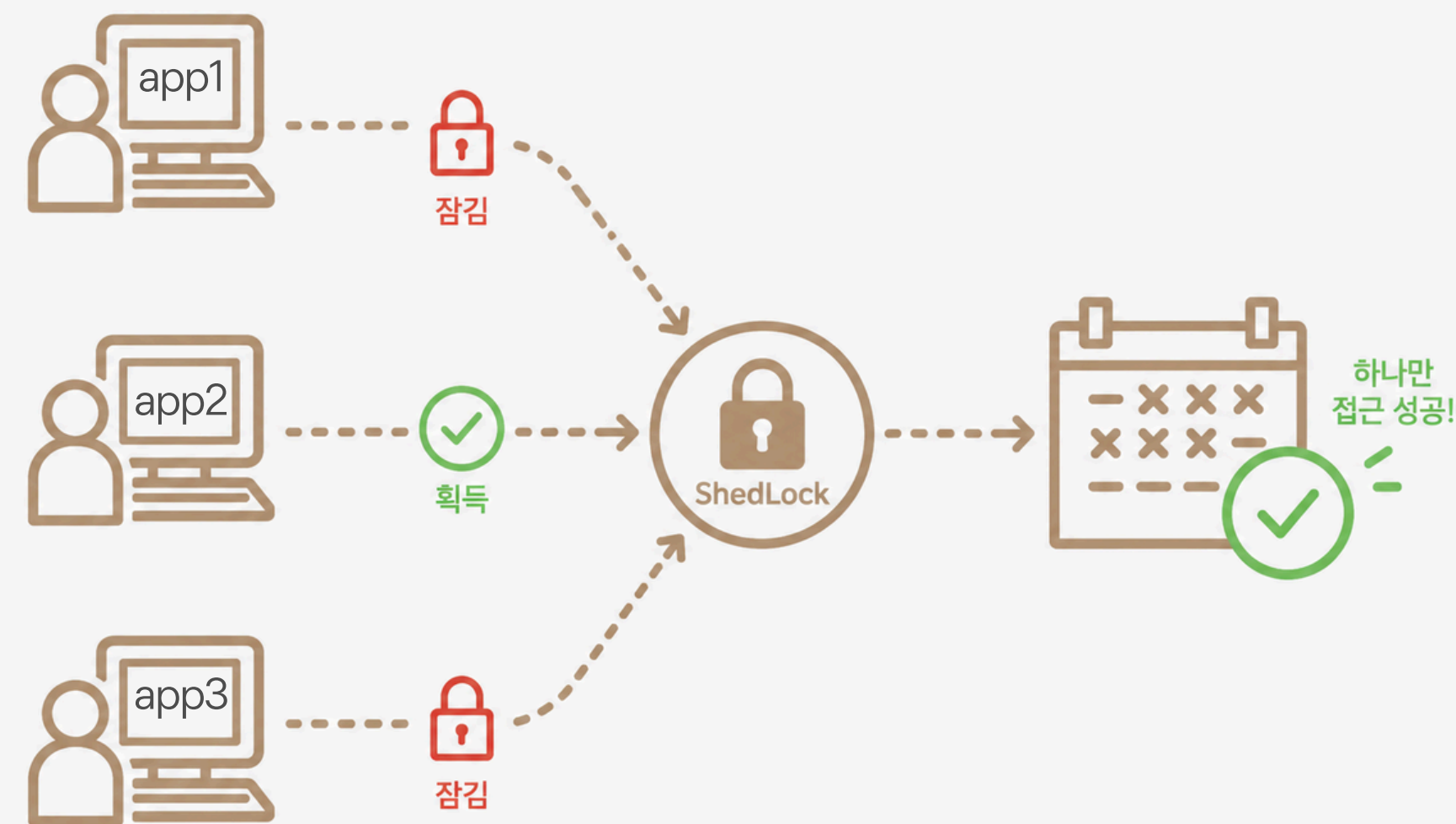
② DistributedLock AOP
→ 재사용성, 가독성 향상

낙관락

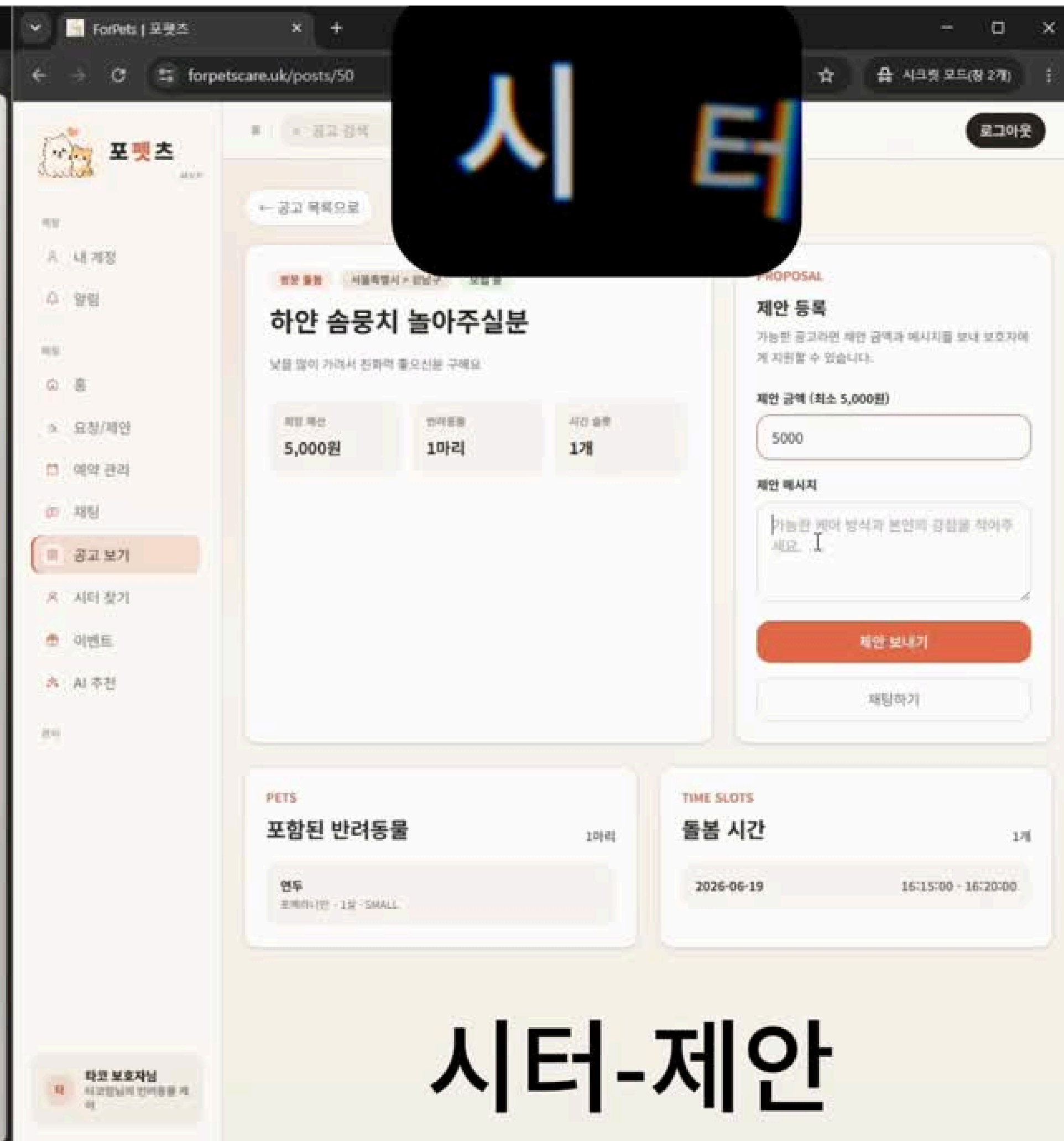
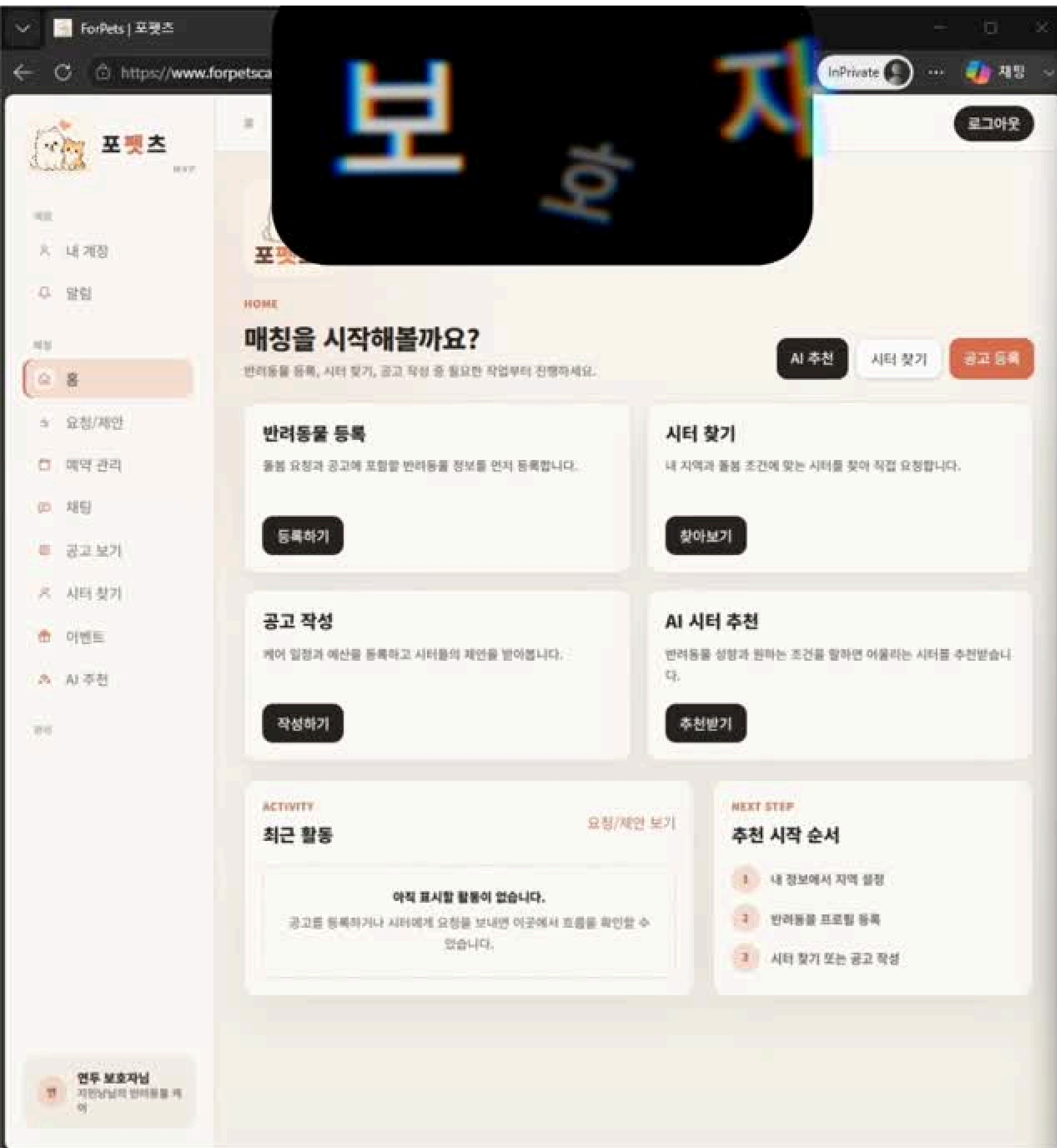
Redis 장애 시에도
Race Condition 방지
→ 이중 방어 구조

ShedLock

스케줄러 전용
중복 실행 방지



시연 영상



시터-제안

트러블슈팅

✓ 쿠폰 임계값 테스트 설계 문제

✓ 문제상황

임계값 측정 중 VU 1,000까지 증가해도
정합성 유지되는 현상 발생

원인: 여러 응답이 섞여서 출력
발급 성공 201
쿠폰 소진 400
락 경합 409

쿠폰 소진 시 락 경합 없이 즉시 응답
VU 증가가 부하로 이어지지 않음

✓ 해결과정

쿠폰 수량 $\geq$ VU 로 재설정
→ 모든 요청이 발급 로직까지 진입

: 쿠폰 소진 변수 제거
응답시간 영향 변수를
락 경합 단일 요인으로 한정

: 임계값 재정의
avg·p95 응답시간의 기울기가
급격히 꺾이는 지점

✓ 회고

부하 테스트 설계 자체가
측정 결과를 왜곡할 수 있음

테스트 시나리오에서
"측정 대상 외 변수"를 제거하는 것이
정확한 임계값 산출의 전제 조건

Spring AOP 체이닝

문제상황

@DistributedLock

@TrackExecutionTime

두 AOP를 같은 메서드에 적용

→ IllegalStateException 발생

매칭 + 파라미터 바인딩을 동시 요청

: 두 번째 Advice 진입 시점에

JoinPointMatch 매핑 정보 누락

→ 바인딩 실패

해결과정

1. argNames 직접 지정 시도

→ 근본 원인은 체이닝 중 매핑 누락

→ 동일 오류 발생, **실패**

2. 바인딩 자체 제거

@annotation(풀 클래스명) 형태 채택

→ Spring에는 매칭만 요청

+ **Reflection**으로 직접 조회

→ 런타임에 어노테이션 인스턴스 추출

결론

JoinPointMatch 바인딩 의존 제거

→ 체이닝 시 누락 이슈 원천 차단

@annotation() → 매칭 + 바인딩

@annotation(클래스명) → 매칭만

Advice 2개 이상 체이닝 시

Reflection 직접 조회 방식이 안전

✓ Graceful Degradation 트러블슈팅

✓ 문제상황

Spring Cache는
**Redis 장애 시 예외를
그대로 던져**, 캐시와 무관한 조회 API
까지 500에러가 발생함.

✓ 해결과정

CacheErrorHandler를 구현한
**GracefulDegradationCacheErrorHan
dler**로 캐시의 기능인
GET / PUT / EVICT / CLEAR
실패를 log.warn으로 삼키고 **DB 직접
조회로 폴백**
로그에 [CACHE_DEGRADED] 키워드를
상수로 박아 필터링·알람 연동을 쉽게 했다.

✓ 배운 점

캐시는 성능 수단이지 필수 인프라
가 아니다.
**Redis가 죽으면 서비스는 느려질
수는 있어도 멈추면 안 된다.**
장애 시 동작을 명시적으로 설계해
야 "캐시 때문에 전체가 죽는" 사고
를 막는다.

✓ Redis Streams Consumer Group

✓ 문제상황

Redis Stream에는 메시지가 쌓였지만, Consumer Group에 **Consumer가 등록되지 않음**

=> 그 결과 메시지가 소비되지 않아 DB 저장과 SSE 알림 전송이 **누락**

✓ 해결과정

1. **Consumer 시작 시점을 SmartLifecycle**로 분리하여 애플리케이션 시작 시 반드시 초기화 하도록 수정

2. **MKSTREAM**으로 Stream이 없을 때도 Group을 생성
BUSYGROUP은 재시작 상황으로 예외 처리

✓ 배운 점

Redis Stream에 메시지가 쌓였다고 전체 알림 흐름이 정상인 것은 아니었음

비동기 메시지 구조는
발행 → 저장 → 소비 → DB 반영 → 실시간 전송을 단계별로 나누어 확인해야함

✓ 배포 환경에서 RAG 검색이 실패한 문제 해결

✓ 문제상황

로컬에서는 AI 시터 추천과
리뷰 근거 표시가 정상 동작

하지만 배포 환경에서는
AI 요청 시 network error 발생

서버 로그 확인 결과
Qdrant 요청이 **localhost:6333**
으로 나가며
Connection refused 발생

✓ 해결과정

서버 로그에서 Qdrant 요청이
localhost:6333으로 나가는 것을 확인

컨테이너 내부의 localhost는
Qdrant가 아닌
Spring Boot 자신을 가리킴

그래서 Qdrant 접속 주소를 localhost에서
http://qdrant:6333 으로
변경해 재배포 함

✓ 회고

문제는 배포 후 해결했지만
배포 전에도 충분히
막을 수 있었던 문제였음

로컬에서 Spring Boot만
직접 실행하며 검증했고
배포와 동일한 Docker Compose
환경 검증이 부족했음

이후에는 **환경 변수, Profile, API Key,**
컨테이너 네트워크를
배포 체크리스트로 확인해야 함



회고

- ✓ 최길중** 쿠폰 동시성 제어를 구현하며 락 전략마다 장애 패턴과 DB 커넥션 영향이 달라진다는 걸 직접 체감했고, 기술 선택이 단순한 구현 문제가 아니라 정책과 운영 안정성까지 고려해야 하는 설계 문제임을 배웠습니다.
- ✓ 권지원** 메인 비즈니스 로직을 구현하며 스케줄링, 동시성, 모니터링을 편하게 넣을 수 있는 구조를 생각했습니다. 단지 기능을 동작하게 하는 것이 아닌 어떻게 구조를 짜고 기획을 해야하는가를 직접 설명할 수 있게 된 부분에서 마지막 프로젝트를 잘 마무리 한 것 같습니다!
- ✓ 박영수** Redis Streams, Pub/Sub, SSE, EC2 멀티 인스턴스 구조를 적용하면서 알림이 안정적으로 전달되기 위해 어떤 흐름이 필요한지 배웠습니다. 단순히 "돌아가는 코드"를 넘어서 배포, 리소스, 네트워크까지 운영에서 필요한 것들까지 함께 생각하는 백엔드 경험을 얻어서 좋았습니다.
- ✓ 선경안** 성능개선(캐싱/인덱싱) 부분을 진행하며 구현된 기능들이나 DB조회 속도 등을 분석하고 이를 토대로 개선해가는 과정이 있었습니다. 성능개선시 실제 운영쪽 데이터나 상황을 고려하여 진행해야 결과를 신뢰할 수 있다는 점을 배웠습니다. 조회나 리뷰 기능을 포함해 초반 정책설계나 ADR을 위한 학습 등 과정이 쉽지는 않았지만 하나씩 알아가는 기쁨을 경험할 수 있어 좋았습니다.
- ✓ 이지민** 역할은 나누었지만 예약, 결제, 알림, AI 기능이 서로 연결되어 있었기 때문에 API 명세와 정책을 지속적으로 맞추는 과정이 중요했습니다. 이번 프로젝트를 통해 단순 기능 구현을 넘어 도메인 간 연결, 상태 전이, 배포 환경, 장애 대응까지 고려하는 백엔드 설계의 중요성을 느꼈습니다.

Q&A

경청해주셔서 감사합니다.

집사조일체

최길중 | 권지원 | 박영수 | 선경안 | 이지민

배포

<https://forpetscare.uk/>



스파르타 내일배움캠프
최종 프로젝트

GitHub

<https://github.com/Housekeeper-for-pets>